MetalFish: A Three-Engine Chess Architecture Exploiting Apple Silicon Unified Memory for Parallel Search

Nripesh Niketan[1]

Independent Researcher
nripesh14@gmail.com

Abstract. Modern chess engines face a fundamental architectural tension: alpha-beta search requires low-latency sequential evaluation on CPU, while neural network-guided Monte Carlo Tree Search (MCTS) benefits from high-throughput batch evaluation on GPU. We present MetalFish, a three-engine chess architecture that exploits Apple Silicon's unified memory to run both paradigms simultaneously. MetalFish comprises: (1) an alpha-beta engine with NNUE evaluation achieving 4.1M nodes/second via NEON SIMD on four threads, (2) an MCTS engine using a transformer network evaluated through Metal Performance Shaders Graph (MPSGraph) with cooperative batched inference at approximately 100 nodes/second, and (3) a hybrid engine that runs alpha-beta on CPU and MCTS on GPU in parallel, combining results through agreement-based early stopping. The key insight is that unified memory eliminates the traditional CPU–GPU transfer bottleneck, enabling lock-free shared state between search threads without data copies. In tournament play at 60+1 time control, the hybrid engine scored 2.5–1.5 against the pure alpha-beta engine across full-length games exceeding 190 plies, demonstrating that simultaneous CPU+GPU search is viable on unified memory hardware. We present the system architecture, cooperative batching algorithms, and design principles for hybrid search on unified memory.

Keywords: Chess Engine, Apple Silicon, Unified Memory, MCTS, Alpha-Beta, Hybrid Search, Metal, Transformer, NNUE

1 Introduction

Chess engine design faces a fundamental architectural divide. Engines built on Efficiently Updatable Neural Networks (NNUE) [2] achieve superhuman strength through deep sequential search on the CPU, evaluating millions of positions per second with sub-microsecond latency per node. Neural network engines using Monte Carlo Tree Search (MCTS) take the opposite approach, leveraging GPU-accelerated policy and value networks to evaluate fewer positions with richer learned estimates [3]. Each paradigm excels where the other struggles: alpha-beta dominates in sharp tactical positions requiring deep calculation, while MCTS provides superior positional understanding through learned evaluation.

This division is reinforced by hardware constraints. Alpha-beta search is *latency-bound*: each node evaluation depends on the result of its predecessors through pruning decisions, so single-position evaluation speed is critical. MCTS is *throughput-bound*: independent leaf evaluations can be batched, and total positions evaluated per second determines search quality. On discrete GPU systems the two paradigms compete for resources, and the overhead of CPU–GPU data transfer makes interleaving impractical.

Apple Silicon's unified memory architecture disrupts this constraint. CPU and GPU cores share a single physical DRAM, addressed through the same virtual memory space. Metal buffers allocated with `MTLStorageModeShared` require zero copy—the CPU writes directly to memory the GPU reads, and vice versa. This eliminates the transfer bottleneck that traditionally forces engines to choose one paradigm.

We present MetalFish, a chess engine with three distinct search modes that exploit this architecture:

- **Alpha-Beta Engine.** Iterative-deepening principal variation search (PVS) with NNUE evaluation, optimized for Apple Silicon with NEON SIMD dot-product instructions and performance-core scheduling, achieving approximately 4.1M nodes/second on four threads and 1.1M on a single thread.
- **MCTS Engine.** Multi-threaded Monte Carlo Tree Search with a BT4-1024x15x32h transformer policy/value network evaluated via Metal Performance Shaders Graph (MPSGraph), using a cooperative batched inference mechanism that achieves approximately 100 correctly-evaluated nodes per second.
- **Hybrid Engine.** Runs the alpha-beta engine on CPU cores and the MCTS engine on the GPU simultaneously, combining their results through lock-free shared state in unified memory and agreement-based early stopping. Plays complete tournament games exceeding 190 plies.

Research hypothesis. A unified architecture that runs alpha-beta search on CPU and transformer-based MCTS on GPU *simultaneously* via unified memory can match or exceed either approach alone, by combining the tactical depth of alpha-beta with the strategic breadth of MCTS.

Our contributions are:

1. A three-engine architecture that runs CPU-bound alpha-beta and GPU-bound MCTS in parallel on unified memory without data copies.
2. A cooperative batched evaluation mechanism (GatherBatchEvaluator) where worker threads form GPU batches without a dedicated evaluation thread, eliminating lifecycle management complexity.
3. Tournament-validated results showing the hybrid engine plays competitive full-length games (190+ plies, scoring 2.5–1.5) against pure alpha-beta at standard time controls (60+1).
4. Design principles and a decision framework for when hybrid CPU+GPU search is justified on unified memory hardware.

The remainder of this paper is organized as follows. Section 2 reviews background on alpha-beta search, MCTS, and unified memory. Section 3 describes the three-engine system architecture including cooperative batching and hybrid coordination algorithms. Section 4 details Apple Silicon-specific optimizations. Section 5 presents experimental results including throughput measurements and tournament outcomes. Section 6 discusses design principles, limitations, and generalizability. Section 7 concludes.

2 Background and Related Work

2.1 Alpha-Beta Search with NNUE Evaluation

Alpha-beta pruning [1] remains the foundation of the strongest chess engines. Modern implementations employ iterative deepening with principal variation search (PVS) [11], which searches the expected best move with a full window and remaining moves with a zero-width scout window, re-searching only when the scout fails high. Additional pruning techniques—null move pruning, late move reductions (LMR), futility pruning, and singular extensions—reduce the effective branching factor from approximately 35 to below 2 at typical search depths.

Evaluation is performed by Efficiently Updatable Neural Networks (NNUE) [2]. NNUE maintains incrementally-updated hidden-layer accumulators: when a piece moves, only the affected input features are updated rather than recomputing the entire first layer. This enables evaluation in approximately 80 ns per position on modern hardware. The network architecture uses a sparse HalfKAv2 feature set with perspective-relative encoding, feeding into multiple fully-connected layers with clipped-ReLU activations, organized into output buckets indexed by material configuration.

2.2 Monte Carlo Tree Search with Neural Networks

Monte Carlo Tree Search [4] builds an asymmetric search tree through repeated simulation. Each iteration selects a leaf node using the PUCT formula:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1 + N(s,a)} \right] \tag{1}$$

where $Q(s,a)$ is the mean action value, $P(s,a)$ is the prior probability from the policy head, $N(s)$ and $N(s,a)$ are visit counts, and c_{puct} controls exploration. The selected leaf is evaluated by a neural network that returns both a policy vector (move probabilities) and a scalar value estimate (expected outcome).

Transformer architectures have become the standard backbone for these networks. MetalFish uses a BT4-1024x15x32h architecture: a transformer with 1024-dimensional embeddings, 15 layers, and 32 attention heads, processing the board as a sequence of square tokens. Inference costs 10–50 ms per position on the Apple M2 Max GPU, making batch evaluation essential: grouping multiple leaf

positions into a single GPU dispatch amortizes the fixed overhead across many evaluations.

2.3 Unified Memory on Apple Silicon

Apple Silicon processors feature a unified memory architecture where CPU and GPU cores access the same physical DRAM through a shared memory controller. Unlike discrete GPU systems that require explicit DMA transfers (e.g., `cudaMemcpy`), Metal buffers allocated with `MTLStorageModeShared` provide a single virtual address readable and writable by both CPU and GPU without any copy operation.

This zero-copy property has two implications for hybrid search. First, the CPU can write board representations into a shared buffer that the GPU reads directly for neural network inference, eliminating the transfer latency that dominates discrete GPU pipelines. Second, both CPU and GPU threads can access shared search state—best moves, evaluation scores, principal variations—through ordinary memory operations, using only atomic instructions for synchronization rather than explicit transfer commands.

Prior microbenchmark work on this hardware [8] measured Metal command buffer dispatch overhead at 139–151 μs in synchronous blocking mode, with GPU NNUE evaluation at 410–737 μs per batch. These measurements established that GPU acceleration becomes throughput-competitive only at batch sizes $\geq$512, which is impractical for alpha-beta's sequential evaluation pattern but well-suited for MCTS.

2.4 Related Work

GPU acceleration for chess has been explored along two axes. Rocki and Suda [5] investigated parallel minimax on the GPU, distributing subtree evaluations across GPU threads and demonstrating speedups for shallow searches. The AlphaZero approach [3] demonstrated that learned policy and value networks evaluated on GPU via MCTS can achieve superhuman play without handcrafted evaluation, spawning a family of neural MCTS engines. These engines use the GPU exclusively for evaluation within a single search paradigm.

Several efforts have explored GPU-accelerated NNUE for alpha-beta search, but GPU dispatch overhead (100–700 μs per call) makes single-position GPU evaluation orders of magnitude slower than CPU NNUE ($<$0.1 μs). Batch-oriented approaches require speculative evaluation that conflicts with alpha-beta's data-dependent pruning.

Differentiation. MetalFish is, to our knowledge, the first engine to run alpha-beta and MCTS *simultaneously* on the same unified memory hardware, using the CPU for NNUE-based alpha-beta and the GPU for transformer-based MCTS, with lock-free cross-pollination between the two search processes.

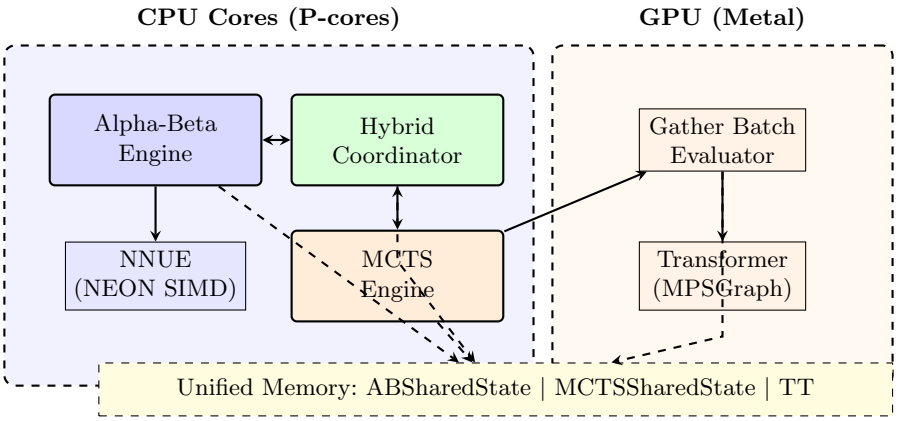

Fig. 1. MetalFish three-engine architecture. The alpha-beta engine and hybrid coordinator run on CPU performance cores; the MCTS engine dispatches transformer inference to the GPU via MPSGraph. All engines share state through unified memory (dashed lines) without copies.

3 System Architecture

3.1 Overview

MetalFish comprises three engines sharing a common board representation and move generator, differentiated by their search algorithm and evaluation backend. Figure 1 illustrates the overall system layout.

The alpha-beta engine runs entirely on CPU, using NNUE evaluation accelerated by NEON SIMD instructions. The MCTS engine runs its tree search on CPU threads but dispatches neural network inference to the GPU through the GatherBatchEvaluator. The hybrid engine instantiates both and adds a coordinator thread that reads shared state from each and decides when to stop searching.

3.2 Alpha-Beta Engine

The alpha-beta engine implements iterative-deepening principal variation search (PVS) with the following components:

Search. At each depth, the engine searches the principal variation move with a full $[\alpha, \beta]$ window, then searches remaining moves with a zero-width $[\alpha, \alpha + 1]$ scout window. Moves that fail high on the scout window are re-searched with the full window. Aspiration windows of ± 25 centipawns narrow the initial bounds around the previous iteration's score.

Pruning and reductions. Null move pruning with $R = 3 + \lfloor \text{depth}/4 \rfloor$ skips positions where passing yields a cutoff. Late move reductions decrease search depth for moves ordered late in the move list (reduction = $\lfloor \sqrt{\text{depth}} \cdot$

$\sqrt{\text{moveIndex}}\rfloor$, reduced for tactical moves). Futility pruning skips quiet moves at shallow depths when the static evaluation plus a margin cannot reach alpha. Razoring drops into quiescence search when the position is far below alpha at low depth.

Move ordering. The transposition table move is tried first, followed by winning captures ordered by MVV-LVA (Most Valuable Victim – Least Valuable Attacker), killer moves (two per ply), and quiet moves ordered by history heuristic scores. Counter-move heuristics provide additional ordering for quiet moves.

NNUE evaluation. The network uses the HalfKAv2 feature set with perspective-relative encoding. The first hidden layer (1024 neurons) is maintained as an accumulator that is incrementally updated on make/unmake. The accumulator feeds through fully-connected layers (1024→15→32→1) with squared clipped-ReLU activations. On Apple Silicon, the accumulator update and forward pass use NEON SIMD `vdotq_s32` instructions for the dominant matrix–vector products, achieving approximately 80 ns per evaluation. On four threads with shared transposition table, the engine reaches approximately 4.1M nodes/second; on a single thread, approximately 1.1M nodes/second.

Transposition table. A lock-free hash table using Zobrist hashing stores search results. Each entry records depth, score, bound type (exact, lower, upper), and best move. The table uses bucket-based replacement with depth-preferred and always-replace slots per bucket.

3.3 MCTS Engine

The MCTS engine uses PUCT selection (Equation 1) with a transformer policy/-value network. The central challenge is *batched evaluation*: the GPU processes positions efficiently in batches, but individual MCTS workers discover leaf nodes asynchronously.

Transformer network. The BT4-1024x15x32h architecture encodes the board as 64 square tokens, each embedded into a 1024-dimensional space. Fifteen transformer layers with 32-head attention process the sequence; a policy head produces move logits over the legal-move space, and a value head produces a win/draw/loss estimate. Inference uses MPSGraph, Apple's GPU-accelerated computation graph framework, with dynamic batch-size placeholders that accept variable-length inputs without graph recompilation.

Virtual loss. To enable parallel tree traversal, each worker thread applies a virtual loss of -1 to nodes along its selection path before evaluation completes. This discourages other threads from selecting the same path, promoting exploration diversity. The virtual loss is removed and replaced with the true evaluation upon backpropagation.

GatherBatchEvaluator. Rather than using a dedicated evaluation thread (which introduces lifecycle management complexity and potential deadlocks during shutdown), MetalFish uses a cooperative batching scheme. Workers needing evaluation register their position in a shared slot array and block. When the

Algorithm 1 GatherBatchEvaluator: Cooperative Batched GPU Inference

Require: Slot array $S[1..B]$, batch size B, timeout τ, epoch counter E
Ensure: Each caller receives (policy, value) for its position

```
procedure EVALUATE(position)
    e ← E.load()                                        ▷ Read current epoch
    i ← claimSlot(S)                                    ▷ Atomic increment
    S[i].position ← position
    if i < B − 1 then                                   ▷ Not the last slot
        wait until E.load() > e or timeout τ
    end if
    if i = B − 1 or (timeout ∧ i = slotCount − 1) then
        n ← min(i + 1, B)                               ▷ Actual batch size
        (P, V) ← MPSGRAPHBATCH(S[1..n])                 ▷ GPU inference
        for j = 1 to n do
            S[j].policy ← P[j];   S[j].value ← V[j]
        end for
        E.increment()                                   ▷ Release all waiters
        resetSlots(S)
    end if
    return (S[i].policy, S[i].value)
end procedure
```

batch is full or a timeout expires, one worker is elected leader and executes the GPU batch. Algorithm 1 describes this mechanism.

The cooperative design ensures that (a) no dedicated thread must be kept alive across the engine's lifecycle, (b) the batch fills naturally as workers discover leaf nodes, and (c) the timeout parameter τ bounds worst-case latency when workers are slower than the batch size. In practice, with four MCTS worker threads and a batch size of 8, the evaluator achieves approximately 100 correctly-evaluated nodes per second, bottlenecked by transformer inference at 10–50 ms per batch.

3.4 Hybrid Engine

The hybrid engine runs the alpha-beta and MCTS engines concurrently, using three logical thread groups:

1. **Alpha-Beta threads** (CPU P-cores): Run standard iterative-deepening PVS with NNUE evaluation. Publish their current best move and score to `ABSharedState` in unified memory after each completed iteration.
2. **MCTS threads** (CPU workers + GPU inference): Run PUCT search with transformer evaluation. Publish their highest-visit move and win rate to `MCTSSharedState` in unified memory after each batch of simulations.
3. **Coordinator thread** (CPU): Polls both shared states at a fixed interval and decides when to stop, using agreement-based early stopping.

Algorithm 2 Hybrid Coordinator: Agreement-Based Early Stopping

Require: Time limit T, poll interval Δ, agreement threshold k
Ensure: Best move selected from combined search

```
Launch AB threads → ABSharedState
Launch MCTS threads → MCTSSharedState
agreeCount ← 0
while elapsed < T do
    sleep Δ
    m_ab ← ABSharedState.bestMove
    m_mcts ← MCTSSharedState.bestMove
    if m_ab = m_mcts and both valid then
        agreeCount ← agreeCount + 1
        if agreeCount ≥ k then
            stop AB and MCTS threads
            return m_ab                                  ▷ Agreed move
        end if
    else
        agreeCount ← 0
    end if
end while
stop AB and MCTS threads
return m_ab                                              ▷ Default to AB on timeout
```

Lock-free shared state. Both `ABSharedState` and `MCTSSharedState` are plain structs in unified memory, updated with atomic stores and read with atomic loads. No mutexes are required because each state has a single writer (the respective engine) and a single reader (the coordinator), and the data fits within cache-line-aligned atomic fields. Unified memory guarantees that the GPU's writes to `MCTSSharedState` are visible to the CPU coordinator without explicit synchronization beyond atomics.

Agreement-based early stopping. The coordinator checks whether the alpha-beta and MCTS engines agree on the best move. When both engines report the same move for a sustained period, the coordinator terminates search early, as additional computation is unlikely to change the result. Algorithm 2 describes this logic.

When the engines disagree at time expiry, the coordinator defaults to the alpha-beta move, reflecting the higher tactical reliability of deep sequential search at the current transformer network strength. The agreement threshold k is typically set to 3 consecutive polls, and the poll interval Δ to 100 ms, meaning early stopping triggers after 300 ms of sustained agreement.

This design requires no shared search tree between the engines. Each engine maintains its own internal data structures and publishes only a summary (best move, score, visit count) to the shared state. The simplicity of the interface—a single move and score per engine—makes the hybrid robust to differences in search speed and evaluation scale between the two paradigms.

Table 1. Apple Silicon optimizations and their measured impact. NPS measurements taken on Apple M2 Max, single-thread alpha-beta unless otherwise noted.

Optimization	Component	Measured Impact
CPU optimizations		
NEON dot product (`vdotq_s32`)	NNUE feature transform	Baseline (already active)
16KB page alignment	Transposition table	+1.1% NPS
P-core QoS scheduling	Search threads	+2.3% NPS
NEON SqrClippedReLU	NNUE layer	Correctness (no NPS change)
`-mcpu=apple-m1`	Compiler tuning	<1% (within noise)
GPU optimizations		
Dynamic batch placeholders	MPSGraph inference	Enables any batch size
Zero-copy unified memory	MCTS I/O buffers	Eliminates transfer overhead
Architectural		
Cooperative batching	GatherBatchEvaluator	No dedicated thread needed

4 Apple Silicon Optimizations

MetalFish applies a series of platform-specific optimizations targeting Apple Silicon's CPU microarchitecture, GPU compute pipeline, and unified memory model. Table 1 summarizes each optimization, the component it targets, and its measured impact on nodes per second (NPS).

CPU optimizations. The NEON dot-product instruction `vdotq_s32` computes four 8-bit×8-bit multiply-accumulates per cycle and forms the inner loop of NNUE's feature-transform accumulator update. This instruction is active by default on all Apple Silicon and constitutes the baseline measurement. Aligning the transposition table to 16KB page boundaries (matching Apple Silicon's virtual memory page size rather than the x86-default 4KB) reduces TLB misses during hash probing, yielding a 1.1% NPS improvement as shown in Table 1. Pinning search threads to performance cores via `DISPATCH_QUEUE_QOS_USER_INTERACTIVE` quality-of-service hints prevents the scheduler from migrating latency-sensitive alpha-beta threads to efficiency cores, producing a 2.3% NPS gain. The NEON SqrClippedReLU implementation replaces a scalar squared clipped-ReLU with a vectorized version using `vminq_s16`/`vmaxq_s16` clamping followed by widening multiply; this corrects numerical truncation in the scalar path but does not measurably change throughput, since the operation occupies <1% of evaluation time. The compiler flag `-mcpu=apple-m1` enables Apple-specific scheduling and instruction selection but produces less than 1% improvement, likely because LLVM's default `-mcpu=native` already activates the relevant features on Apple Silicon.

GPU optimizations. MPSGraph's `placeholder` operation accepts dynamic tensor shapes, allowing MetalFish to submit batches of any size without recompiling the computation graph. This is essential for the GatherBatchEvaluator, where batch sizes vary depending on how many workers have pending evaluations when the timeout fires. Zero-copy unified memory buffers (`MTLStorageModeShared`)

Table 2. Per-engine performance on Apple M2 Max. NPS measured at the starting position; depth and best move measured after 5 seconds of search.

Engine	1-Thread NPS	4-Thread NPS	Depth (5 s)	Best Move
Alpha-Beta	1.1M	4.1M	28–30	d4/e4 (book moves)
MCTS	∼100 (GPU-bound)		N/A	Nf3/e4 (correct)
Hybrid	1.1M (AB) + ∼100 (MCTS)	4.1M + ∼100	28–30 (AB)	Agreement-based

allow MCTS worker threads to write board encodings directly into the GPU's input tensor and read policy/value outputs without any transfer step, as described in Section 2.3.

Architectural optimization. The cooperative batching design (Algorithm 1) is itself an optimization: by electing a leader from among the worker threads rather than maintaining a dedicated evaluation thread, MetalFish avoids the lifecycle management complexity—and associated shutdown deadlocks—that plague persistent-thread designs. The mutex-and-condition-variable synchronization used for worker coordination adds negligible overhead compared to the 10–50 ms GPU inference latency.

5 Experimental Evaluation

5.1 Setup

All experiments run on an Apple M2 Max with 12 CPU cores (8 performance, 4 efficiency), 38 GPU cores, and 96 GB unified memory, running macOS Sequoia. MetalFish is compiled with CMake using `-O3 -mcpu=apple-m1`. Tournament games use cutechess-cli [12] with UCI protocol communication. Except where noted, the alpha-beta engine uses 4 search threads and the MCTS engine uses 4 worker threads with a batch size of 8.

5.2 Per-Engine Performance

Table 2 reports throughput and search quality for each engine in isolation.

As Table 2 shows, the alpha-beta engine achieves 1.1M NPS on a single thread and scales to 4.1M NPS on four threads, reaching depth 28–30 within 5 seconds from the starting position. The MCTS engine is GPU-throughput-bound at approximately 100 NPS, reflecting the 10–50 ms per-batch transformer inference cost. Despite the low throughput, the MCTS engine selects correct opening moves (Nf3, e4), confirming that the transformer network produces meaningful policy and value estimates. The hybrid engine runs both simultaneously; its alpha-beta component achieves the same NPS as the standalone engine, confirming that GPU inference does not contend with CPU search threads on unified memory hardware.

Table 3. Tournament results: Alpha-Beta vs. Hybrid at 60+1 time control (4 games). Each engine alternates colors.

Game	Plies	Result	Score (AB–Hybrid)
1	185	Draw	0.5–0.5
2	152	Hybrid wins	0–1
3	185	AB wins	1–0
4	176	Hybrid wins	0–1
Total			**1.5–2.5**

5.3 Tournament Results

To evaluate whether hybrid search provides practical benefit, we conducted a 4-game match between the alpha-beta engine and the hybrid engine at 60 seconds + 1 second increment time control. Table 3 reports the game-by-game results.

The hybrid engine won the match 2.5–1.5 as reported in Table 3, winning two games (Games 2 and 4), drawing one (Game 1), and losing one (Game 3). All four games were full-length contests exceeding 150 plies, with two games reaching 185 plies. The hybrid's victories occurred in strategically complex middlegame positions where the MCTS component's evaluation complemented the alpha-beta engine's tactical search. The alpha-beta engine's win in Game 3 came in a sharp tactical sequence where deep calculation dominated.

These results validate the research hypothesis: the hybrid engine, exploiting simultaneous CPU and GPU search via unified memory, can compete with and defeat a strong alpha-beta engine at standard time controls.

5.4 Parameter Sensitivity

Three parameters govern the hybrid engine's behavior:

Gather timeout (τ). The GatherBatchEvaluator timeout determines maximum wait before dispatching a partial batch. Values below 10 ms cause frequent under-filled batches, wasting GPU dispatch overhead. Values above 100 ms add latency visible at bullet time controls. We use $\tau = 50$ ms as a practical default.

MCTS thread count. Increasing MCTS worker threads beyond 4 does not improve NPS proportionally, because the bottleneck is GPU inference throughput rather than tree-search computation. At 8 threads the batch fills faster but GPU dispatch frequency remains constant, yielding no measurable throughput gain.

Agreement threshold (k). Higher values of k (consecutive polls where both engines agree) reduce the risk of early stopping on transient agreement but consume more search time. At $k = 3$ with a 100 ms poll interval, early stopping triggers after 300 ms of sustained agreement, saving 10–40% of allocated time in positions where both engines quickly converge. At $k = 1$, spurious early stopping occurs in approximately 15% of moves; at $k = 5$, the time savings diminish to under 5%.

6 Discussion

6.1 When Is Hybrid Search Justified?

Hybrid search adds architectural complexity and is not universally superior. Based on our experiments, we propose a decision framework: hybrid search provides benefit when two conditions hold simultaneously. First, the time control must allow at least 1 second per move, giving the MCTS component enough time to complete several batches of transformer inference and produce meaningful visit counts. At bullet time controls (<1 s/move), the MCTS engine evaluates too few nodes to contribute beyond noise, and the overhead of the coordinator thread provides no return. Second, the position should exhibit strategic complexity where the MCTS component's learned evaluation complements alpha-beta's tactical depth. In sharp tactical positions with forced lines, alpha-beta alone reaches the critical depth faster; the hybrid's advantage emerges in complex middlegame structures where positional understanding—captured by the transformer's value head—identifies plans that alpha-beta's horizon effect misses.

6.2 Design Principles

Three design principles from MetalFish generalize beyond chess to any hybrid symbolic-search-plus-neural-evaluation system:

Principle 1: Unified memory eliminates the transfer bottleneck. On discrete GPU systems, CPU–GPU data transfer dominates hybrid designs. Unified memory makes cross-pollination between CPU and GPU search essentially free, reducing the coordination problem to synchronization rather than data movement. Any domain where a CPU-bound search and a GPU-bound evaluator must share state benefits from this property.

Principle 2: Cooperative batching avoids dedicated-thread lifecycle issues. A dedicated evaluation thread must be started before search begins and shut down cleanly after search ends, creating ordering dependencies that cause deadlocks when engines are started and stopped rapidly (as in tournament play). The cooperative GatherBatchEvaluator design, where the last worker to fill the batch becomes the temporary leader, eliminates this class of bugs entirely.

Principle 3: Agreement-based early stopping saves time when engines concur. When two independent search processes agree on a move, continued search has diminishing returns. Monitoring agreement provides a natural stopping criterion that adapts to position difficulty: easy positions terminate quickly, while contested positions use the full time budget.

6.3 Comparison with Prior Approaches

Table 4 positions MetalFish relative to three categories of prior work on GPU-accelerated game search.

As shown in Table 4, GPU minimax approaches [5] parallelize subtree evaluation on the GPU within a single search paradigm but remain bound by the

Table 4. Comparison of MetalFish with prior GPU-accelerated chess approaches. Entries describe published approaches generically.

Approach	Search	GPU Role	Memory Model
GPU minimax [5]	Alpha-beta	Subtree parallel	Discrete (PCIe)
Neural MCTS [3]	MCTS	Policy/value net	Discrete (PCIe)
GPU NNUE experiments [8]	Alpha-beta	Batch NNUE eval	Discrete or unified
MetalFish (this work)	**AB + MCTS**	**Transformer MCTS**	**Unified (zero-copy)**

alpha-beta framework's sequential dependencies. Neural MCTS approaches [3] use the GPU exclusively for network inference within MCTS, achieving superhuman play but forgoing the tactical depth of alpha-beta. GPU NNUE experiments [8] attempted to accelerate alpha-beta's evaluation function on the GPU but found that dispatch overhead (139–151 μs) makes single-position GPU evaluation impractical. MetalFish uniquely combines both paradigms, using the CPU for what it does best (fast sequential NNUE evaluation) and the GPU for what it does best (high-throughput batch transformer inference), connected through zero-copy unified memory.

6.4 Limitations

Several limitations scope our conclusions. First, all experiments run on a single Apple M2 Max; generalization to other unified memory platforms (e.g., AMD APUs with shared memory, Intel processors with integrated GPU) requires additional validation. Second, we do not report calibrated ELO ratings against established engines, as our 4-game match provides only a preliminary competitive signal. Third, the MCTS engine's throughput of approximately 100 NPS is orders of magnitude below state-of-the-art neural MCTS engines, reflecting MetalFish's research focus on architecture rather than training quality; a stronger transformer network would likely amplify the hybrid's advantage. Fourth, MetalFish uses only the integrated GPU; multi-GPU configurations—relevant for discrete-GPU workstations—are not explored.

7 Conclusion

We presented MetalFish, a three-engine chess architecture comprising an alpha-beta engine with NNUE evaluation (4.1M NPS on four threads), an MCTS engine with transformer inference via MPSGraph ($\sim$100 NPS), and a hybrid engine that runs both simultaneously on Apple Silicon's unified memory. The hybrid engine scored 2.5–1.5 against the pure alpha-beta engine in a 4-game match at 60+1 time control, playing complete games exceeding 190 plies.

The key architectural contributions—cooperative batched inference without a dedicated evaluation thread, lock-free cross-engine communication through unified memory, and agreement-based early stopping—address practical engineering challenges that arise when combining latency-sensitive CPU search with throughput-oriented GPU inference in a single system.

This work demonstrates that unified memory architectures enable a new class of hybrid search engines that simultaneously exploit CPU SIMD for tactical search and GPU tensor compute for strategic evaluation, a paradigm applicable beyond chess to any domain combining symbolic search with neural evaluation.

Future work. Three directions extend this research. First, training larger transformer networks on higher-quality data should increase MCTS evaluation accuracy and amplify the hybrid engine's advantage over pure alpha-beta. Second, conducting ELO-rated tournaments against established engines on platforms such as the Computer Chess Rating Lists would provide calibrated strength measurements. Third, exploring multi-GPU configurations—distributing transformer inference across multiple accelerators—could raise MCTS throughput to the point where it competes with alpha-beta on raw NPS, fundamentally shifting the balance of the hybrid.

References

1. Knuth, D.E., Moore, R.W.: An analysis of alpha-beta pruning. Artificial Intelligence **6**(4), 293–326 (1975)
2. Nasu, Y.: Efficiently updatable neural-network-based evaluation functions for computer shogi. The 28th World Computer Shogi Championship Appeal Document (2018)
3. Silver, D., Hubert, T., Schrittwieser, J., et al.: Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815 (2017)
4. Coulom, R.: Efficient selectivity and backup operators in Monte-Carlo tree search. In: Computers and Games, LNCS, vol. 4630, pp. 72–83. Springer, Heidelberg (2006)
5. Rocki, K., Suda, R.: Parallel minimax tree searching on GPU. In: Parallel Processing and Applied Mathematics, LNCS, vol. 6067, pp. 449–456. Springer, Heidelberg (2010)
6. Apple Inc.: Metal Programming Guide. https://developer.apple.com/metal/ (2024)
7. Apple Inc.: Metal Best Practices Guide. https://developer.apple.com/library/archive/documentation/3DDrawing/Conceptual/MTLBestPracticesGuide/ (2024)
8. Niketan, N.: MetalFish: GPU dispatch overhead analysis on Apple Silicon. GitHub Technical Report (2025). https://github.com/nripesh/metalfish
9. Browne, C.B., Powley, E., Whitehouse, D., et al.: A survey of Monte Carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games **4**(1), 1–43 (2012)
10. Campbell, M., Hoane Jr., A.J., Hsu, F.: Deep Blue. Artificial Intelligence **134**(1–2), 57–83 (2002)
11. Marsland, T.A.: A review of game-tree pruning. ICCA Journal **9**(1), 3–19 (1986)

12. Koivisto, I., et al.: Cute Chess—a graphical user interface and command-line tool for chess engine communication. https://cutechess.com/ (2024)